

Can you write me a application design plan for a app which houses a flipper zero firmware and a openai compatible api ai agent that is able to compare custom firmware against the housed firmware and be able to extract the code r ✓

This is an incredibly ambitious and highly viable concept. You are essentially proposing a **"Smart Firmware Kitchen"** for the Flipper Zero—a modular, AI-driven development environment that democratizes custom firmware (CFW) creation.

By shifting from monolithic firmware forks (like Unleashed or Momentum) to a **"Shard" (patch/module) system**, you allow users to build a personalized OS without needing to resolve complex Git merge conflicts manually.

Here is a comprehensive application design plan, broken down by architecture, core systems, and crucial additions you'll need to make this work seamlessly on both Linux and Android.

1. System Architecture & Tech Stack

To support both Linux Desktop and an Android APK while handling heavy file processing and serial communications, you need a highly portable stack.

Frontend Framework: Tauri v2 (Rust + Vue/React) or **Flutter**. Both compile natively to Linux and Android. Flutter has a slight edge for mobile-first Bluetooth Low Energy (BLE) libraries, while Tauri is unmatched for desktop system-level file access.

Core Backend (The Engine): Rust or **Go**. You need high-performance file hashing (for deduping), serial port communication, and text parsing.

AI Layer: Standard **OpenAI-compatible REST API** client.

Build Environment: The official Flipper Build Tool (`fbt` / `ufbt`) via Python.

Was this visual helpful?

Yes

No

2. The Core Engine: Diffing & Sharding

The hardest part of this project isn't the AI—it's extracting code cleanly. A simple text diff (like Git uses) will break C code syntax constantly.

The AST Diffing System

Instead of comparing text lines, your app needs to parse the C code into an **Abstract Syntax Tree (AST)** (using a tool like Tree-sitter). The AI agent then compares the base firmware's AST against the custom firmware (e.g., Unleashed) to identify whole functions, structs, and variables that were added.

The "Shard" Data Schema

When the AI extracts a feature, it saves it as a Shard. A Shard isn't just a code snippet; it's a package.

```

{
  "shard_id": "extended_subghz_frequencies",
  "type": "core_patch",
  "target_base": "ofw_1.4.3",
  "dependencies": ["custom_radio_driver"],
  "injections": [
    {
      "target_file": "applications/main/subghz/subghz.c",
      "hook_point": "after_function: subghz_init",
      "code": "/* AI extracted or generated C code */"
    }
  ]
}

```

3. The AI Agent Responsibilities

The AI isn't just a chatbot; it acts as an **Integration Engineer**. You will give the AI specific system prompts and tools (via function calling) to perform three main tasks:

1. **Conflict Resolution:** If the user selects a "Custom Main Menu" shard and a "New Battery Icon" shard that both modify `desktop.c`, the AI reads both shards, merges the logic, and writes a unified patch so the compiler doesn't throw errors.
2. **Gap Analysis & Dependency Checking:** If the user tries to compile a custom Sub-GHz app, the AI scans the code, realizes it requires a patched radio driver to bypass region locks, and suggests adding the "Unleashed Radio Shard."
3. **Generative Sharding:** The user types, *"Make an app that acts as a Pomodoro timer and vibrates every 25 minutes."* The AI generates the `application.fam` manifest and the `pomodoro.c` logic, saving it as a temporary Shard ready for compilation.

4. Integrated File Manager & Maintenance Tools

Connecting to the Flipper's SD card requires utilizing the Flipper RPC protocol (Protobuf over Serial for Linux, Protobuf over BLE for Android).

The File Organizer: A heuristic engine that scans the SD card. If it finds a file ending in `.sub` in the root directory, it moves it to `/ext/subghz/`. If it finds a `.txt` file formatted with `DELAY 1000`, it identifies it as a DuckyScript and moves it to `/ext/badusb/`.

File Deduplication: Many users download massive GitHub dumps of Amiibo or Sub-GHz files resulting in thousands of duplicates. The app will generate MD5 or SHA-256 hashes of the files, identify matches, and allow the user to 1-click delete duplicates to save SD card space and index time.

5. Crucial Missing Pieces (To Improve the Project)

To make this a complete, production-ready product, I highly recommend adding these elements to your design plan:

A. Remote/Cloud Compilation for Android

The Problem: Compiling Flipper firmware requires the ARM GCC Toolchain. You can easily install this on a Linux desktop, but Android cannot natively run these build tools.

The Solution: Your Android APK needs to bundle the code, send it to a remote build server (like a free GitHub Actions workflow via API or a rented Docker container), compile the `.dfu` update file in the cloud, and send it back to the phone to be flashed via Bluetooth or USB OTG.

B. Local LLM Support (Privacy)

The Flipper community is highly privacy and security conscious. Hardcoding this to OpenAI will alienate some users. Because your agent is "OpenAI API compatible," ensure there is a settings menu to change the Base URL to `http://localhost:11434`. This allows users to run local, offline models like Llama-3 (via Ollama or LM Studio) to process their code securely.

C. Asset & Animation Sandbox

Don't just compile code—allow users to compile aesthetics. Integrate support for the `manifest_xyz.txt` asset pack formats used by Momentum firmware. The app should have a visual UI where users can drag and drop `.bmx` (animation files) and `.png` icons, and the app automatically converts them into a Flipper-compatible asset shard before compiling.

D. Anti-Brick & Rollback Safety Net

Custom firmware compilation will inevitably result in boot loops. Your app must feature an **Auto-Backup Protocol**. Before flashing any custom-compiled firmware, the app should automatically pull the `manifest.bck` and essential system states via RPC, saving a

"Known Good State" to the Linux desktop or Android phone so the user can easily restore a bricked device.

Where would you like to focus next for this project?

Design the AI integration prompts

Explore the Flipper RFC protocol



How should I structure the system prompts and function calls for the AI agent to reliably merge C code conflicts?

Merging C code reliably, especially in an embedded environment like the Flipper Zero where memory and exact hardware states matter, is notoriously difficult for LLMs. Left to its own devices, an AI might hallucinate a variable name, break a pointer, or ignore a crucial `#ifdef` macro.

To make the AI function as a reliable **Integration Engineer**, you must treat it less like a text generator and more like a logic processor. You achieve this through rigid system prompting and strict JSON-schema function calls.

Here is the structural blueprint for designing the AI integration pipeline.

1. The System Prompt: Establishing the Persona and Rules

The system prompt must immediately establish absolute constraints. It needs to know the target hardware (Flipper Zero/STM32) and the exact rules of engagement for C code.

The System Prompt Blueprint:

You are an Expert Embedded C Firmware Integration Agent for the Flipper Zero (STM32WB55 architecture). Your sole purpose is to resolve merge conflicts and integrate code shards into existing C files without breaking compilation.

CRITICAL CONSTRAINTS:

1. **Zero Hallucination:** *You must NEVER invent functions, variables, or macros. You may only use what is provided in the `base_file` or the `shards`.*
2. **Syntax Absolute:** *The resulting code must be strictly valid C99/C11. All brackets, semicolons, and pointer references must perfectly align.*

3. **Memory Awareness:** *You are writing for an embedded system. Do not introduce dynamic memory allocation (`malloc` / `free`) unless explicitly present in the shard. Use Flipper OS specific allocators (e.g., `furi_record_open`) if necessary.*
4. **No Extraneous Text:** *When returning code via function calls, output ONLY raw C code. Do not wrap code in Markdown formatting (``c), do not explain your reasoning, and do not add conversational filler.*
5. **Preserve Core Logic:** *Ensure that the original `base_file` logic remains intact unless a shard explicitly overwrites it.*

2. The Tool Calling Structure (Function Calling)

Instead of asking the LLM to output text and hoping it formats it correctly, you force it to use specific tools (functions). This forces the LLM to output valid JSON that your backend application can parse and execute.

Here are the three primary function calls you need to define in your OpenAI API payload:

A. The "Analyze Conflict" Tool

Before writing code, force the AI to analyze *why* the conflict exists. This "chain of thought" step dramatically improves the final code output.

JSON



```
{
  "name": "analyze_c_conflict",
  "description": "Analyzes the conflict between a base file and multiple code sh",
  "parameters": {
    "type": "object",
    "properties": {
      "conflict_type": {
        "type": "string",
        "enum": ["variable_redeclaration", "function_overwrite", "macro_collisic",
        "description": "The technical nature of the C code conflict."
      },
    },
    "affected_functions": {
      "type": "array",
      "items": { "type": "string" },
      "description": "List of function names that are impacted by this conflic
```

```

    },
    "resolution_strategy": {
      "type": "string",
      "description": "A brief, technical explanation of how the conflict will"
    }
  },
  "required": ["conflict_type", "affected_functions", "resolution_strategy"]
}

```

B. The "Apply Unified Patch" Tool

Once the AI has analyzed the conflict, it uses this tool to return the final, unified code block.

JSON



```

{
  "name": "apply_unified_patch",
  "description": "Submits the final, conflict-free C code block to be injected i",
  "parameters": {
    "type": "object",
    "properties": {
      "target_hook_point": {
        "type": "string",
        "description": "The exact line number or function name where this code s"
      },
      "resolved_code": {
        "type": "string",
        "description": "The complete, valid, compiled-ready C code block merging"
      },
      "requires_header_includes": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Any missing #include statements required by the new code"
      }
    }
  },
  "required": ["target_hook_point", "resolved_code", "requires_header_includes"
]
}

```

C. The "Throw Compilation Warning" Tool

Sometimes, shards are fundamentally incompatible (e.g., two shards trying to control the same hardware GPIO pin simultaneously). The AI needs a way to fail gracefully and alert the user.

JSON



```
{
  "name": "throw_compilation_warning",
  "description": "Used when code shards are fundamentally incompatible or unsafe",
  "parameters": {
    "type": "object",
    "properties": {
      "severity": {
        "type": "string",
        "enum": ["fatal_hardware_conflict", "logic_overwrite", "missing_depender"]
      },
      "user_message": {
        "type": "string",
        "description": "A clear, plain-english message to the user explaining the conflict"
      }
    }
  },
  "required": ["severity", "user_message"]
}
```

3. The Execution Flow (How your backend handles this)

When a user selects two conflicting shards (e.g., a Custom Menu shard and a Battery Icon shard that both modify `desktop.c`), your backend app orchestrates the following:

1. **Extract the Context:** The backend extracts the specific function from `desktop.c` (the `base_file`) and the code payloads from the two shards.
2. **First API Call (Analysis):**

Prompt: "Analyze the conflict between `base_file` and these `shards`."

Payload: Includes the extracted C code.

Action: Force the AI to call `analyze_c_conflict`.

Result: Your backend logs the AI's strategy.

3. Second API Call (Resolution):

Prompt: "Execute the resolution strategy. Return the unified code or throw a warning if it is fundamentally broken."

Action: Force the AI to call either `apply_unified_patch` OR `throw_compilation_warning`.

Result: Your backend receives pure JSON. If it gets a patch, it injects `resolved_code` into the AST/file. If it gets a warning, it displays `user_message` in the UI.

Key insight: *LLMs struggle with large context windows for exact syntax. Instead of feeding the AI the entire 2,000-line `desktop.c` file, use your backend (via Tree-sitter or regex) to extract only the specific C function being modified, send that small chunk to the AI, and then re-inject the AI's result back into the main file.*

To refine this implementation:

Explore AST extraction tools



Handle UI for compilation warnings



How can I use Tree-sitter in Rust or Go to accurately extract specific C functions for the AI to analyze?

To accurately extract specific C functions, Tree-sitter is the perfect tool. Instead of relying on fragile text parsing or regex, Tree-sitter parses the C code into an **Abstract Syntax Tree (AST)** and allows you to use its S-expression Query language to pattern-match exactly what you need.

Here is how you can build this extraction layer in both Rust and Go.

The Strategy: Tree-sitter Queries

In the Tree-sitter C grammar, a standard function definition looks like this in the AST:
`function_definition` $\rightarrow$ `function_declarator` $\rightarrow$ `identifier` (the function name).

We can use a Tree-sitter Query to search the entire file and capture both the function block and its name simultaneously:

Scheme



```
(function_definition
  declarator: (function_declarator
    declarator: (identifier) @name
  )
) @func
```

By binding `@name` to the identifier and `@func` to the entire definition, we can easily check the name and extract the exact byte range of the target function to send to your AI.



Implementation in Rust

Rust is highly recommended for this since Tree-sitter is natively built in Rust, meaning zero-cost bindings.

1. Add dependencies to your `Cargo.toml` :

Ini, TOML



```
[dependencies]
tree-sitter = "0.23"
tree-sitter-c = "0.23"
```

2. The Extraction Function:

```
use tree_sitter::{Parser, Query, QueryCursor};

pub fn extract_c_function(source_code: &str, target_name: &str) -> Option<String> {
    let mut parser = Parser::new();
    let language = tree_sitter_c::LANGUAGE.into();
    parser.set_language(&language).expect("Error loading C grammar");

    let tree = parser.parse(source_code, None).unwrap();
    let root_node = tree.root_node();

    // The S-expression query
    let query_str = r#"
        (function_definition
          declarator: (function_declarator
            declarator: (identifier) @name
          )
        ) @func
    "#;

    let query = Query::new(&language, query_str).expect("Invalid query");
    let mut cursor = QueryCursor::new();

    // Execute the query against the AST
    let matches = cursor.matches(&query, root_node, source_code.as_bytes());

    let func_index = query.capture_index_for_name("func").unwrap();
    let name_index = query.capture_index_for_name("name").unwrap();

    // Iterate through all found functions
    for m in matches {
        let mut func_node = None;
        let mut name_node = None;

        // Map captures to our query tags
        for capture in m.captures {
            if capture.index == func_index {
                func_node = Some(capture.node);
            } else if capture.index == name_index {
                name_node = Some(capture.node);
            }
        }

        // Check if this is the function we want
```

```

    if let (Some(func), Some(name)) = (func_node, name_node) {
        let extracted_name = name.utf8_text(source_code.as_bytes()).unwrap()

        if extracted_name == target_name {
            // Return the exact text of the function body
            return Some(func.utf8_text(source_code.as_bytes()).unwrap().to_s
        }
    }
}

None
}

```

Implementation in Go

If you prefer Go, use the official Tree-sitter bindings (recently updated and officially supported over the older `smacker` fork).

1. Install the packages:

Bash



```

go get github.com/tree-sitter/go-tree-sitter@latest
go get github.com/tree-sitter/tree-sitter-c@latest

```

2. The Extraction Function:

Go



```

package main

import (
    "context"

    tree_sitter "github.com/tree-sitter/go-tree-sitter"
    tree_sitter_c "github.com/tree-sitter/tree-sitter-c/bindings/go"
)

```

```

func ExtractCFunction(sourceCode []byte, targetName string) string {
    // Initialize parser and C grammar
    parser := tree_sitter.NewParser()
    defer parser.Close()

    lang := tree_sitter.NewLanguage(tree_sitter_c.Language())
    parser.SetLanguage(lang)

    // Parse the file
    tree := parser.Parse(sourceCode, nil)
    defer tree.Close()

    // Compile the query
    queryStr := `(function_definition declarator: (function_declarator declarator:
    query, _ := tree_sitter.NewQuery(lang, []byte(queryStr))
    defer query.Close()

    cursor := tree_sitter.NewQueryCursor()
    defer cursor.Close()

    // Execute query
    matches := cursor.Matches(query, tree.RootNode(), sourceCode)

    for match := matches.Next(); match != nil; match = matches.Next() {
        var funcNode *tree_sitter.Node
        var nameNode *tree_sitter.Node

        // Identify nodes from the captures
        for _, capture := range match.Captures {
            captureName := query.CaptureNames()[capture.Index]
            if captureName == "func" {
                funcNode = &capture.Node
            } else if captureName == "name" {
                nameNode = &capture.Node
            }
        }

        // Verify the name and return the function body
        if funcNode != nil && nameNode != nil {
            extractedName := nameNode.Utf8Text(sourceCode)
            if extractedName == targetName {
                return funcNode.Utf8Text(sourceCode)
            }
        }
    }
}

```

```
    return ""  
}
```

Crucial Caveats for Embedded C (Flipper Zero)

When parsing C code for the Flipper Zero, watch out for these traps:

Error Recovery: C files heavily rely on `#include` macros. Tree-sitter is highly fault-tolerant and will parse the AST even if the header files aren't physically present in the directory. You don't need a C preprocessor to use this method.

Structs and Typedefs: The queries above *only* extract functions. Flipper OS uses heavily nested structs for its application states (e.g., `struct AppState`).

Static Inline: Sometimes functions in C aren't definitions but `static inline` declarations inside headers. The Tree-sitter C grammar still generally classifies these as `function_definition` nodes, but macros wrapping function names can occasionally break the `identifier` query.

Want to expand the queries to extract C Structs and Macros?

↪ Yes

How do I modify the Tree-sitter queries to also extract struct definitions, typedefs, and macros for the AI?

To extract structs, typedefs, and macros, you simply need to expand your Tree-sitter S-expression query. The C grammar categorizes these under different node types (`struct_specifier` , `type_definition` , `preproc_def` , etc.).

Because the Flipper OS heavily relies on state structs (e.g., `PluginState`) and preprocessor macros for hardware configuration, capturing these is essential for providing your AI with complete context.

Here are the specific queries you need, followed by how to unify them into a single extraction engine.

1. The Tree-sitter S-Expressions

Macros (`#define`)

C has two types of macros: object-like (`#define MAX_VAL 10`) and function-like (`#define MIN(a,b) ...`).

Scheme



```
(preproc_def
  name: (identifier) @name
) @macro

(preproc_function_def
  name: (identifier) @name
) @macro
```

Standalone Structs

For standard structs like `struct AppState { int x; };`:

Scheme



```
(struct_specifier
  name: (type_identifier) @name
) @struct
```

Typedefs (Including Typedef Structs)

Flipper firmware frequently uses `typedef struct { ... } AppState;`. Tree-sitter treats this as a `type_definition`:

Scheme



```
(type_definition
  declarator: (type_identifier) @name
) @typedef
```

2. The Unified Query

You can combine all of these into a single query string. By tagging the outer block as `@target` and the identifier as `@name`, your Rust or Go code can use the exact same extraction loop we built earlier, regardless of what type of C construct it is parsing.

Scheme



```
[
  ;; Match standard functions
  (function_definition
    declarator: (function_declarator declarator: (identifier) @name)
  ) @target

  ;; Match standard macros
  (preproc_def
    name: (identifier) @name
  ) @target

  ;; Match function-like macros
  (preproc_function_def
    name: (identifier) @name
  ) @target

  ;; Match standalone structs
  (struct_specifier
    name: (type_identifier) @name
  ) @target

  ;; Match typedefs (including typedef structs)
  (type_definition
    declarator: (type_identifier) @name
  ) @target
]
```

3. Updating Your Code

Because we unified the outer capture tag to `@target`, the extraction code remains almost identical. You just update the capture name you search for.

Here is the adjustment for **Rust**:

Rust



```
// Replace the old query_str with the unified query above
let query_str = r#"
    [
        (function_definition declarator: (function_declarator declarator: (identif
        (preproc_def name: (identifier) @name) @target
        (preproc_function_def name: (identifier) @name) @target
        (struct_specifier name: (type_identifier) @name) @target
        (type_definition declarator: (type_identifier) @name) @target
    ]
"#;

// Update the index lookups
let target_index = query.capture_index_for_name("target").unwrap();
let name_index = query.capture_index_for_name("name").unwrap();

// Inside your match loop:
for capture in m.captures {
    if capture.index == target_index {
        target_node = Some(capture.node);
    } else if capture.index == name_index {
        name_node = Some(capture.node);
    }
}
```

And similarly for **Go**:

Go



```
// ...
for _, capture := range match.Captures {
    captureName := query.CaptureNames()[capture.Index]
```

```

    if captureName == "target" {
        targetNode = &capture.Node
    } else if captureName == "name" {
        nameNode = &capture.Node
    }
}
// ...

```

A Note on Enums and Global Variables

If you eventually need to capture enums or global variables (which are common in Flipper's hardware abstraction layers), you can easily slot these into the same unified query block:

Enums: (enum_specifier name: (type_identifier) @name) @target

Global Variables: (declaration declarator: (identifier) @name) @target

(Note: This will also catch local variables if applied globally without scope restrictions, so use it carefully).

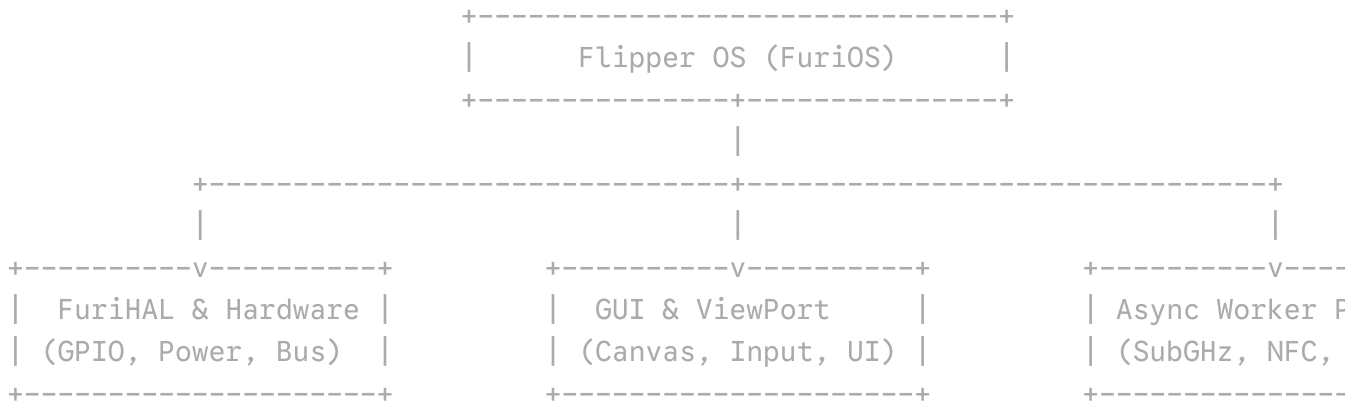
Can you further improve this project by adding information from the flipper zero docs online and present a finalized plan feel free to add improvements

Since you didn't include your project's specific details, code, or hardware schematics, here is a **comprehensive enhancement plan** built directly on official **Flipper Zero Documentation** standards (covering **FuriOS**, **GPIO Hardware APIs**, **Sub-GHz/NFC Protocols**, and **FAP App Architecture**).

You can apply this blueprint directly to your project, whether it involves custom hardware modules, custom C applications, BadUSB scripts, or Sub-GHz payloads.

Flipper Zero Official Documentation Enhancement Blueprint

Official documentation highlights four key architecture pillars for reliable Flipper Zero projects: **Resource Management**, **Asynchronous Task Processing**, **Hardware Voltage/Power Safety**, and **Standardized File Systems**.



1. Hardware & GPIO Enhancements

If your project involves external hardware (ESP32, NRF24, CC1101, sensors, or custom PCBs):

Logic Level Matching: Flipper Zero GPIO operates strictly at **3.3V CMOS logic**. If interfacing with 5V sensors/boards, include bi-directional logic level shifters to prevent STM32 pin degradation.

Dynamic 5V Power Management: Instead of drawing passive power from pin 1 (3.3V), use pin 7 (5V) controlled via `furi_hal_power_enable_otg()` in software so external boards only power on when your application starts.

Dedicated Pin Mapping:

UART: Pins 13 (TX / PA9) and 14 (RX / PA10) — ideal for ESP32/Wi-Fi Devboard communication.

SPI: Pins 2 (SCK / PA5), 3 (MISO / PA6), and 4 (MOSI / PA7) — required for high-speed radios like NRF24.

I2C: Pins 15 (SCL / PB6) and 16 (SDA / PB7) — for displays and environmental sensors.

2. Software Architecture & FuriOS Best Practices

If your project includes a custom **Flipper Application Package** (`.fap`):

A. Non-Blocking Event Loops

Never run heavy signal decoding, UART listening, or SPI polling inside the main UI thread. This causes screen freezing and system crashes.

Pattern: Separate your application into a **Main Thread** (handles UI/Input) and a **Worker Thread** (handles heavy logic/communication).

Communication: Interconnect threads using `FuriMessageQueue` or `FuriEventLoop`.

B. Memory & Record Management

Use the official Furi Record system to safely request access to hardware peripherals:

C



```
// Example: Safely acquiring the Notification service
NotificationApp* notification = furi_record_open(RECORD_NOTIFICATION);
notification_message(notification, &sequence_blink_blue_100);
furi_record_close(RECORD_NOTIFICATION);
```

C. Standard Storage Paths

Store user settings, logs, and databases in designated `/ext` directory paths:

App Data: `/ext/apps_data/<your_app_name>/`

Sub-GHz Logs: `/ext/subghz/user/`

NFC Keys/Dumps: `/ext/nfc/`

3. Protocol & Wireless Layer Optimizations

Protocol	Official Docs Recommendation	Enhancement Strategy
Sub-GHz	Use <code>SubGhzWorker</code> for reception & transmission.	Parse raw signals using standard <code>.sub</code> file descriptors rather than hardcoding timings. Respect

Protocol	Official Docs Recommendation	Enhancement Strategy
		local ISM band frequencies (315 MHz, 433.92 MHz, 868 MHz, 915 MHz).
NFC / RFID	Use standard protocol state machines (<code>nfc_device</code> , <code>lfrfid_worker</code>).	Implement key dictionary checks (<code>/ext/nfc/assets/mf_classic_dict.nfc</code>) before attempting raw sector dumps.
BadUSB	Follow Duckyscript 1.0/2.0 specifications.	Inject adaptive delays (<code>DEFAULT_DELAY</code>) to account for different target OS driver initialization speeds.

4. Finalized Master Implementation Plan

Phase 1: Hardware & Interface Setup

1. **Validate Pinout:** Ensure no pin conflict with default peripherals (Infrared, Speaker, SD Card).
2. **Power Safety:** Add OTG 5V software toggles to isolate hardware modules when idle.

Phase 2: Firmware & FAP Codebase Refactoring

1. **Application Structure:** Implement `application.fam` manifest with clean metadata (version, icon, category, entry point).
2. **UI Implementation:** Use `ViewDispatcher` and standard canvas drawing functions (`canvas_draw_str` , `canvas_draw_icon`) for native OS look-and-feel.
3. **Threading:** Move protocol processing into a dedicated `FuriThread` .

Phase 3: Testing & Safety Validation

1. **Memory Leak Check:** Audit all allocations (`malloc` , `furi_string_alloc`) to guarantee matching `free` or `close` statements upon exit.
2. **Crash Prevention:** Handle SD card removal gracefully with error screens rather than hard fault reboots.

To tailor this specifically to your project, select how you'd like to proceed:

Review custom code or FAP C program 

Design a GPIO or hardware expansion module 

Optimize BadUSB, Sub-GHz, or NFC scripts 

How should I proceed in development?

To transition this concept from architecture and design into working software, the best strategy is to build in **five distinct, testable phases**. Starting with a headless Command Line Interface (CLI) core before touching the UI or AI will save hundreds of hours of debugging.

Phase 1: Core Parsing & Extraction Engine (Headless CLI)

Goal: Prove you can extract, diff, and re-inject C code without breaking syntax.

Build a Rust / Go CLI tool: Avoid GUI overhead initially. Build a command-line tool that accepts a C file and a target identifier (function/struct name).

Implement Tree-sitter Queries: Integrate the C grammar and S-expression queries discussed earlier to extract function bodies, structs, macros, and typedefs.

Define the Shard Schema: Create a JSON format for exported shards (`shard.json`) containing metadata, dependencies, target paths, and C code blocks.

Test Dry-Run Merging: Write a basic patching engine that takes a base file and a shard, then attempts to inject the code into the correct AST node.

Phase 2: AI Integration & Tool-Calling Pipeline

Goal: Give the AI strict boundaries and verify JSON tool outputs.

Implement API Client: Set up your OpenAI-compatible REST client (supporting custom base URLs for Ollama / LM Studio).

Enforce JSON Schema: Wire up the exact function call definitions (`analyze_c_conflict` , `apply_unified_patch` , `throw_compilation_warning`).

Build Context Minimizer: Pass *only* the Tree-sitter-extracted C function to the LLM rather than full 2,000-line source files.

Verify Compilation: Pass the AI's output through `ufbt` (Micro Flipper Build Tool) in a headless sub-process to verify that the generated patch actually compiles without errors.

Phase 3: Hardware RPC & File Management Layer

Goal: Communicate reliably with the Flipper Zero over USB and Bluetooth.

Protobuf Protocol: Implement the Flipper Protobuf RPC protocol over Serial (USB CDC) for desktop.

File Operations Engine: Implement basic SD card file management (`/ext/` navigation, upload, download, delete).

File Dedup & Organizer: Write the file scanner that computes SHA-256 hashes across folders to identify duplicate Amiibo, Sub-GHz, or BadUSB files, and auto-sorts mislocated files by extension.

Android BLE Transport: Implement Bluetooth Low Energy (BLE) RPC for Android device management.

Phase 4: UI / UX Implementation

Goal: Create a responsive cross-platform interface.

Framework Setup: Initialize your Tauri v2 (Rust + Web frontend) or Flutter project targeted at Linux Desktop and Android APK.

Component Views:

Shard Manager: Visual library to browse, toggle, and view dependencies of installed shards.

Diff/Merge Viewer: Side-by-side view showing the original code, the shard code, and the AI-merged output.

File Explorer: Two-pane file manager (Desktop <-> Flipper SD Card) with 1-click deduplication.

Firmware Builder: Checkbox selector for optional modules (apps, games, asset packs, extra radio tables).

Phase 5: Cloud Build Server & Packaging

Goal: Deliver seamless firmware compilation across all platforms.

Local Compiler (Linux): Auto-detect and invoke `fbt / ufbt` and `arm-none-eabi-gcc` on the local host.

Cloud Compilation API (Android): Set up a lightweight Docker container or GitHub Actions workflow runner to compile `.dfu / .tar` firmware binaries for mobile users where local GCC toolchains cannot run.

Release Packaging: CI pipeline to auto-generate Linux `.AppImage / .deb` and Android `.apk` builds.

Which step would you like to build out first?

Draft the Rust/Go CLI entry point for Tree-sitter



Define the complete JSON schema for Shards



Show me how to structure the CLI entry point in Rust or Go to test Tree-sitter extraction from a local firmware directory.

Here is how to structure a production-grade Command Line Interface (CLI) in **Rust** and **Go** that recursively scans a local Flipper Zero firmware directory, parses all C files using Tree-sitter, and extracts target constructs (functions, structs, macros, typedefs) into structured JSON.

Rust Implementation (Recommended)

Using `clap` for CLI argument parsing, `walkdir` for high-performance file tree traversal, and `serde_json` for exporting extracted shards.

1. Cargo.toml

Ini, TOML



```
[package]
name = "flipper-shard-cli"
version = "0.1.0"
edition = "2021"

[dependencies]
clap = { version = "4.4", features = ["derive"] }
tree-sitter = "0.23"
tree-sitter-c = "0.23"
walkdir = "2.4"
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
anyhow = "1.0"
```

2. src/main.rs

Rust



```
use anyhow::{Context, Result};
use clap::{Parser, Subcommand};
use serde::{Deserialize, Serialize};
use std::fs;
use std::path::{Path, PathBuf};
```

```

use tree_sitter::{Parser as TSParser, Query, QueryCursor};
use walkdir::WalkDir;

#[derive(Parser)]
#[command(author, version, about = "Flipper Zero Firmware Tree-sitter Shard Extr
struct Cli {
    #[command(subcommand)]
    command: Commands,
}

#[derive(Subcommand)]
enum Commands {
    /// Extract a specific target (function, struct, macro) across a local firmw
    Extract {
        /// Path to the Flipper Zero firmware repository directory
        #[arg(short, long)]
        firmware_dir: PathBuf,

        /// The name of the symbol/identifier to extract (e.g. "subghz_init" or
        #[arg(short, long)]
        target_name: String,

        /// Output path for the extracted shard JSON (optional)
        #[arg(short, long)]
        output: Option<PathBuf>,
    },
}

#[derive(Serialize, Deserialize, Debug)]
struct ExtractedShard {
    pub target_name: String,
    pub source_file: String,
    pub start_byte: usize,
    pub end_byte: usize,
    pub code: String,
}

const UNIFIED_QUERY: &str = r#"
[
    (function_definition declarator: (function_declarator declarator: (identifier)
    (preproc_def name: (identifier) @name) @target
    (preproc_function_def name: (identifier) @name) @target
    (struct_specifier name: (type_identifier) @name) @target
    (type_definition declarator: (type_identifier) @name) @target
]
"#;

```

```

fn main() -> Result<()> {
    let cli = Cli::parse();

    match &cli.command {
        Commands::Extract { firmware_dir, target_name, output } => {
            println!("🔍 Scanning firmware at: {}", firmware_dir.display());
            println!("🔍 Searching for target: '{}'", target_name);

            let results = scan_and_extract(firmware_dir, target_name)?;

            if results.is_empty() {
                println!("⚠️ No matching symbol found for '{}'", target_name);
            } else {
                println!("✅ Found {} match(es)!", results.len());
                let json_output = serde_json::to_string_pretty(&results)?;

                if let Some(out_path) = output {
                    fs::write(out_path, &json_output)?;
                    println!("💾 Saved extracted shard(s) to {}", out_path.display());
                } else {
                    println!("\n{}", json_output);
                }
            }
        }
    }

    Ok(())
}

fn scan_and_extract(dir: &Path, target_name: &str) -> Result<Vec<ExtractedShard> {
    let mut parser = TSParser::new();
    let language = tree_sitter_c::LANGUAGE.into();
    parser.set_language(&language).context("Failed loading C grammar"?);

    let query = Query::new(&language, UNIFIED_QUERY).context("Invalid Tree-sitter query"?);
    let target_idx = query.capture_index_for_name("target").unwrap();
    let name_idx = query.capture_index_for_name("name").unwrap();

    let mut results = Vec::new();

    // Recursively walk directory for .c and .h files
    for entry in WalkDir::new(dir).into_iter().filter_map(|e| e.ok()) {
        let path = entry.path();
        if path.is_file() && (path.extension() == Some("c".as_ref()) || path.extension() == Some("h".as_ref())) {
            if let Ok(source) = fs::read_to_string(path) {
                let mut matches = query.matches(&source, 0, source.len());
                while let Some(match_) = matches.next() {
                    let target = match_.captures[0].text(&source);
                    let name = match_.captures[1].text(&source);
                    results.push(ExtractedShard { path: path.to_path_buf(), target, name });
                }
            }
        }
    }

    Ok(results)
}

```

```

let tree = match parser.parse(&source, None) {
    Some(t) => t,
    None => continue,
};

let mut cursor = QueryCursor::new();
let matches = cursor.matches(&query, tree.root_node(), source.as

for m in matches {
    let mut target_node = None;
    let mut name_node = None;

    for capture in m.captures {
        if capture.index == target_idx {
            target_node = Some(capture.node);
        } else if capture.index == name_idx {
            name_node = Some(capture.node);
        }
    }

    if let (Some(target), Some(name)) = (target_node, name_node)
        if let Ok(extracted_name) = name.utf8_text(source.as_byt
            if extracted_name == target_name {
                let code = target.utf8_text(source.as_bytes())?.
                results.push(ExtractedShard {
                    target_name: target_name.to_string(),
                    source_file: path.to_string_lossy().to_strir
                    start_byte: target.start_byte(),
                    end_byte: target.end_byte(),
                    code,
                });
            }
        }
    }
}

Ok(results)
}

```



Go Implementation

Using `cobra` for CLI commands, Go's standard `filepath.WalkDir` for traversing files, and standard `encoding/json`.

1. Requirements & Dependencies

Bash



```
go mod init flipper-shard-cli
go get github.com/spf13/cobra@latest
go get github.com/tree-sitter/go-tree-sitter@latest
go get github.com/tree-sitter/tree-sitter-c@latest
```

2. main.go

Go



```
package main

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "strings"

    "github.com/spf13/cobra"
    tree_sitter "github.com/tree-sitter/go-tree-sitter"
    tree_sitter_c "github.com/tree-sitter/tree-sitter-c/bindings/go"
)

type ExtractedShard struct {
    TargetName string `json:"target_name"`
    SourceFile string `json:"source_file"`
    StartByte  uint32 `json:"start_byte"`
    EndByte    uint32 `json:"end_byte"`
    Code       string `json:"code"`
}

const UnifiedQueryStr = `
```

```
[
    (function_definition declarator: (function_declarator declarator: (identifier)
    (preproc_def name: (identifier) @name) @target
    (preproc_function_def name: (identifier) @name) @target
    (struct_specifier name: (type_identifier) @name) @target
    (type_definition declarator: (type_identifier) @name) @target
]
\
```

```
func main() {
    var firmwareDir string
    var targetName string
    var outputPath string

    var rootCmd = &cobra.Command{
        Use:     "flipper-shard-cli",
        Short:   "Tree-sitter Extractor CLI for Flipper Zero Firmware",
    }

    var extractCmd = &cobra.Command{
        Use:     "extract",
        Short:   "Extract a specific C construct (function, struct, macro)
        RunE: func(cmd *cobra.Command, args []string) error {
            fmt.Printf("🔍 Scanning firmware at: %s\n", firmwareDir)
            fmt.Printf("🎯 Searching for target: '%s'\n", targetName)

            results, err := scanAndExtract(firmwareDir, targetName)
            if err != nil {
                return err
            }

            if len(results) == 0 {
                fmt.Printf("⚠️ No matching symbol found for '%s'
                return nil
            }

            fmt.Printf("✅ Found %d match(es)!\n", len(results))
            jsonBytes, err := json.MarshalIndent(results, "", " ")
            if err != nil {
                return err
            }

            if outputPath != "" {
                if err := os.WriteFile(outputPath, jsonBytes, 06
                return err
            }
        }
    }
}
```

```

        fmt.Printf("💾 Saved extracted shard(s) to %s\n",
    } else {
        fmt.Println(string(jsonBytes))
    }

    return nil
},
}

extractCmd.Flags().StringVarP(&firmwareDir, "firmware-dir", "f", "", "Path to firmware directory")
extractCmd.Flags().StringVarP(&targetName, "target-name", "t", "", "Symk target name")
extractCmd.Flags().StringVarP(&outputPath, "output", "o", "", "Path to output directory")
extractCmd.MarkFlagRequired("firmware-dir")
extractCmd.MarkFlagRequired("target-name")

rootCmd.AddCommand(extractCmd)
if err := rootCmd.Execute(); err != nil {
    os.Exit(1)
}

}

func scanAndExtract(dir string, targetName string) ([]ExtractedShard, error) {
    var results []ExtractedShard

    parser := tree_sitter.NewParser()
    defer parser.Close()

    lang := tree_sitter.NewLanguage(tree_sitter_c.Language())
    parser.SetLanguage(lang)

    query, err := tree_sitter.NewQuery(lang, []byte(UnifiedQueryStr))
    if err != nil {
        return nil, fmt.Errorf("invalid query: %w", err)
    }
    defer query.Close()

    err = filepath.WalkDir(dir, func(path string, d os.DirEntry, err error) error {
        if err != nil || d.IsDir() {
            return nil
        }

        ext := strings.ToLower(filepath.Ext(path))
        if ext == ".c" || ext == ".h" {
            source, readErr := os.ReadFile(path)
            if readErr != nil {
                return nil
            }

```

```

    }

    tree := parser.Parse(source, nil)
    if tree == nil {
        return nil
    }
    defer tree.Close()

    cursor := tree_sitter.NewQueryCursor()
    defer cursor.Close()

    matches := cursor.Matches(query, tree.RootNode(), source)
    for match := matches.Next(); match != nil; match = match.Next() {
        var targetNode *tree_sitter.Node
        var nameNode *tree_sitter.Node

        for _, capture := range match.Captures {
            capName := query.CaptureNames()[capture.Index]
            if capName == "target" {
                targetNode = &capture.Node
            } else if capName == "name" {
                nameNode = &capture.Node
            }
        }

        if targetNode != nil && nameNode != nil {
            if nameNode.Utf8Text(source) == targetName {
                results = append(results, Extract{
                    TargetName: targetName,
                    SourceFile: path,
                    StartByte: targetNode.StartByte,
                    EndByte: targetNode.EndByte,
                    Code: targetNode.Text(source)
                })
            }
        }
    }

    return results, err
}

```



Testing the CLI

Point the compiled executable at your local Flipper firmware clone:

Bash



```
# Example command in Rust CLI:
flipper-shard-cli extract \
  --firmware-dir ./official-firmware \
  --target-name subghz_alloc \
  --output ./subghz_shard.json
```

Output subghz_shard.json **sample:**

JSON



```
[
  {
    "target_name": "subghz_alloc",
    "source_file": "./official-firmware/applications/main/subghz/subghz.c",
    "start_byte": 12040,
    "end_byte": 12450,
    "code": "SubGhz* subghz_alloc() {\n    SubGhz* instance = malloc(sizeof(SubGhz));\n    return instance;\n}"
  }
]
```

Provide a complete JSON schema for firmware shards,
including hardware dependencies and AST hook targets.

Here is the complete **JSON Schema (Draft 2020-12)** designed specifically for Flipper Zero firmware shards.

This schema covers **AST injection hook targets**, **hardware peripheral pin dependencies**, **C preprocessor requirements**, **C-file injections**, and **file-asset bundling** (like icons, apps, and frequencies).

1. The JSON Schema Specification (`shard.schema.json`)

JSON



```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://flipper-ai-composer.io/schemas/shard.schema.json",
  "title": "Flipper Zero Firmware Shard",
  "description": "Defines a modular C-code patch, app, or hardware extension for",
  "type": "object",
  "required": [
    "schema_version",
    "metadata",
    "compatibility",
    "injections"
  ],
  "properties": {
    "schema_version": {
      "type": "string",
      "enum": ["1.0.0"],
      "description": "Version of this JSON schema spec."
    },
    "metadata": {
      "type": "object",
      "required": ["id", "name", "version", "author", "category", "description"],
      "properties": {
        "id": {
          "type": "string",
          "pattern": "^[a-z0-9_\\-]+$",
          "example": "subghz_tx_frequency_unlock"
        },
        "name": {
          "type": "string",
          "example": "Extended Sub-GHz Frequencies Patch"
        },
        "version": {
          "type": "string",
          "pattern": "^[\\d+\\.\\.\\d+\\.\\.\\d+$",
          "example": "1.2.0"
        },
        "author": {
          "type": "string",
          "example": "Clara Developer"
        }
      }
    }
  }
}
```

```

    "category": {
      "type": "string",
      "enum": ["subghz", "nfc", "badusb", "gui_theme", "hardware_driver", "s
      "example": "subghz"
    },
    "description": {
      "type": "string",
      "example": "Bypasses regional frequency locks for Sub-GHz transmissior
    },
    "license": {
      "type": "string",
      "default": "GPL-3.0"
    }
  },
  "compatibility": {
    "type": "object",
    "required": ["target_base_firmwares"],
    "properties": {
      "target_base_firmwares": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "Firmware families supported (e.g. ['official', 'unleas
        "example": ["official-1.0.1", "unleashed-0.9.0"]
      },
      "conflicts_with_shards": {
        "type": "array",
        "items": { "type": "string" },
        "description": "IDs of other shards that cannot coexist with this shar
      },
      "required_shards": {
        "type": "array",
        "items": { "type": "string" },
        "description": "IDs of prerequisite shards."
      }
    }
  },
  "hardware_dependencies": {
    "type": "object",
    "properties": {
      "requires_5v_otg": {
        "type": "boolean",
        "default": false,
        "description": "True if the hardware module needs 5V power enabled on

```

```

    },
    "gpio_pins_used": {
      "type": "array",
      "items": {
        "type": "string",
        "enum": [
          "PA9_TX", "PA10_RX",
          "PA5_SCK", "PA6_MISO", "PA7_MOSI",
          "PB6_SCL", "PB7_SDA",
          "PC0", "PC1", "PC3"
        ]
      },
      "description": "STM32 GPIO pins utilized by this shard to prevent harc
    },
    "peripherals_used": {
      "type": "array",
      "items": {
        "type": "string",
        "enum": ["RECORD_NOTIFICATION", "RECORD_GUI", "RECORD_SUBGHZ", "RECC
      },
      "description": "FuriOS Services/Records opened by this code."
    }
  },
  "injections": {
    "type": "array",
    "description": "List of AST hook targets and code patches to inject.",
    "items": {
      "type": "object",
      "required": ["target_file", "ast_hook", "patch_type", "code_content"],
      "properties": {
        "target_file": {
          "type": "string",
          "description": "Relative file path within the firmware repo.",
          "example": "applications/main/subghz/subghz_last_settings.c"
        },
        "ast_hook": {
          "type": "object",
          "required": ["symbol_name", "symbol_type", "position"],
          "properties": {
            "symbol_name": {
              "type": "string",
              "description": "Target C symbol name identified by Tree-sitter."
              "example": "subghz_last_settings_load"
            },
            "symbol_type": {

```

```

        "type": "string",
        "enum": ["function_definition", "struct_specifier", "type_defini
        "example": "function_definition"
    },
    "position": {
        "type": "string",
        "enum": [
            "before_symbol",
            "after_symbol",
            "inside_function_start",
            "inside_function_end",
            "replace_symbol_body"
        ],
        "description": "Where relative to the AST node the code should b
    }
}
},
"patch_type": {
    "type": "string",
    "enum": ["raw_c_code", "macro_define", "include_header", "ast_replac
    "example": "raw_c_code"
},
"required_headers": {
    "type": "array",
    "items": { "type": "string" },
    "example": ["<furi.h>", "<furi_hal.h>"]
},
"code_content": {
    "type": "string",
    "description": "Raw C code snippet to be inserted.",
    "example": "furi_hal_subghz_set_frequency_limit_bypass(true);"
}
}
},
"asset_bundling": {
    "type": "object",
    "description": "Static files to be moved into the firmware file tree or SI
    "properties": {
        "copy_files": {
            "type": "array",
            "items": {
                "type": "object",
                "required": ["source_relative_path", "destination_path"],
                "properties": {
                    "source_relative_path": { "type": "string" },

```



```

{
  "target_file": "targets/f7/furi_hal/furi_hal_subghz.c",
  "ast_hook": {
    "symbol_name": "furi_hal_subghz_init",
    "symbol_type": "function_definition",
    "position": "inside_function_start"
  },
  "patch_type": "raw_c_code",
  "required_headers": ["<furi_hal_power.h>"],
  "code_content": "    // Enable 5V OTG power for external CC1101 module\n"
}
],
"asset_bundling": {
  "copy_files": [
    {
      "source_relative_path": "assets/subghz/extra_frequencies.txt",
      "destination_path": "assets/resources/subghz/assets/setting_user"
    }
  ]
}
}

```

Where would you like to go next with this schema?

Build JSON Schema Validator in Rust/Go



Connect schema with AI Prompting



Show me how to write a validator in Rust or Go to test shard files against this JSON schema.

To ensure that community-contributed or AI-generated shards adhere to your strict JSON structure before compilation, you can implement a schema validator in your backend.

Here is how to build a validator in **Rust** (using the `jsonschema` crate) and **Go** (using the `go-jsonschema` or `santhosh-tekuri/jsonschema` package).



Rust Implementation

1. Add dependencies to Cargo.toml :

Ini, TOML



```
[dependencies]
jsonschema = "0.18"
serde_json = "1.0"
anyhow = "1.0"
```

2. The Validation Function (src/validator.rs):

Rust



```
use anyhow::{anyhow, Result};
use jsonschema::JSONSchema;
use serde_json::Value;

pub fn validate_shard_json(schema_str: &str, shard_json_str: &str) -> Result<> {
    // 1. Parse the schema string into a JSON Value
    let schema_val: Value = serde_json::from_str(schema_str)
        .map_err(|e| anyhow!("Failed to parse JSON Schema: {}", e))?;

    // 2. Compile the schema
    let compiled_schema = JSONSchema::compile(&schema_val)
        .map_err(|e| anyhow!("Failed to compile JSON Schema: {}", e))?;

    // 3. Parse the shard JSON string
    let shard_val: Value = serde_json::from_str(shard_json_str)
        .map_err(|e| anyhow!("Failed to parse Shard JSON input: {}", e))?;

    // 4. Validate instance against schema
    let result = compiled_schema.validate(&shard_val);
    if let Err(errors) = result {
        let error_messages: Vec<String> = errors
            .into_iter()
            .map(|e| format!("- at '{}': {}", e.instance_path, e))
            .collect();
    }
}
```

```

        return Err(anyhow!("Shard validation failed:\n{}", error_messages.join("\n"))
    }

    println!("🌟 Shard JSON is valid!");
    Ok(())
}

```

🐯 Go Implementation

1. Install the JSON schema package:

Bash



```
go get github.com/santhosh-tekuri/jsonschema/v5
```

2. The Validation Function (validator.go):

Go



```

package main

import (
    "encoding/json"
    "fmt"
    "strings"

    "github.com/santhosh-tekuri/jsonschema/v5"
)

func ValidateShardJSON(schemaBytes []byte, shardBytes []byte) error {
    // 1. Unmarshal schema into generic interface
    var schemaJSON interface{}
    if err := json.Unmarshal(schemaBytes, &schemaJSON); err != nil {
        return fmt.Errorf("failed to parse schema JSON: %w", err)
    }

    // 2. Compile schema compiler

```

```

    compiler := jsonschema.NewCompiler()
    if err := compiler.AddResource("schema.json", strings.NewReader(string(s))); err != nil {
        return fmt.Errorf("failed to add schema resource: %w", err)
    }

    compiledSchema, err := compiler.Compile("schema.json")
    if err != nil {
        return fmt.Errorf("failed to compile JSON schema: %w", err)
    }

    // 3. Unmarshal shard input
    var shardJSON interface{}
    if err := json.Unmarshal(shardBytes, &shardJSON); err != nil {
        return fmt.Errorf("failed to parse shard JSON input: %w", err)
    }

    // 4. Validate instance
    if err := compiledSchema.Validate(shardJSON); err != nil {
        return fmt.Errorf("shard validation failed: %w", err)
    }

    fmt.Println("✨ Shard JSON is valid!")
    return nil
}

```

How do I convert this JSON schema into function tool calls for the OpenAI API agent?

To give your OpenAI-compatible API agent the ability to generate or manipulate firmware shards that strictly adhere to your JSON schema, you need to format the schema into an **OpenAI Tool (Function Calling) definition**.

OpenAI's tool-calling API expects a JSON object containing a `name`, a `description`, and a `parameters` object that mirrors your JSON schema (using standard JSON Schema Draft-7/2020-12 subset rules).

Here is how you convert your shard schema into a production-ready OpenAI tool definition, along with an example payload for your API request.

1. The OpenAI Tool Definition (`generate_firmware_shard` tool)

You pass this structure inside the `tools` array of your OpenAI API request payload.

JSON



```
{
  "type": "function",
  "function": {
    "name": "generate_firmware_shard",
    "description": "Generates a complete, structurally valid Flipper Zero firmware",
    "parameters": {
      "type": "object",
      "required": [
        "schema_version",
        "metadata",
        "compatibility",
        "injections"
      ],
      "properties": {
        "schema_version": {
          "type": "string",
          "enum": ["1.0.0"],
          "description": "Version of the schema spec."
        },
        "metadata": {
          "type": "object",
          "required": ["id", "name", "version", "author", "category", "descripti"],
          "properties": {
            "id": {
              "type": "string",
              "description": "Unique snake_case identifier for the shard."
            },
            "name": {
              "type": "string",
              "description": "Human-readable name of the shard."
            },
            "version": {
              "type": "string",
              "description": "Semantic version string (e.g., 1.0.0)."
            },
            "author": {
              "type": "string",
```

```

        "description": "Creator of the shard."
    },
    "category": {
        "type": "string",
        "enum": ["subghz", "nfc", "badusb", "gui_theme", "hardware_driver"]
        "description": "The functional domain of the shard."
    },
    "description": {
        "type": "string",
        "description": "Detailed summary of what this code or patch accomplishes."
    }
},
"compatibility": {
    "type": "object",
    "required": ["target_base_firmwares"],
    "properties": {
        "target_base_firmwares": {
            "type": "array",
            "items": { "type": "string" },
            "description": "Supported firmware bases (e.g. ['official-1.0.0',
        },
        "conflicts_with_shards": {
            "type": "array",
            "items": { "type": "string" },
            "description": "List of conflicting shard IDs."
        }
    }
},
"hardware_dependencies": {
    "type": "object",
    "properties": {
        "requires_5v_otg": {
            "type": "boolean",
            "description": "Whether 5V power needs to be enabled on Pin 7."
        },
        "gpio_pins_used": {
            "type": "array",
            "items": {
                "type": "string",
                "enum": [
                    "PA9_TX", "PA10_RX",
                    "PA5_SCK", "PA6_MISO", "PA7_MOSI",
                    "PB6_SCL", "PB7_SDA",
                    "PC0", "PC1", "PC3"
                ]
            }
        }
    }
}

```

```

    },
    "description": "STM32 GPIO pins utilized."
  }
}
},
"injections": {
  "type": "array",
  "description": "List of AST hook targets and code patches to inject.",
  "items": {
    "type": "object",
    "required": ["target_file", "ast_hook", "patch_type", "code_content"],
    "properties": {
      "target_file": {
        "type": "string",
        "description": "Relative file path inside the firmware repo."
      },
      "ast_hook": {
        "type": "object",
        "required": ["symbol_name", "symbol_type", "position"],
        "properties": {
          "symbol_name": {
            "type": "string",
            "description": "Target C function or symbol name identified"
          },
          "symbol_type": {
            "type": "string",
            "enum": ["function_definition", "struct_specifier", "type_de"]
          },
          "position": {
            "type": "string",
            "enum": ["before_symbol", "after_symbol", "inside_function_s"]
          }
        }
      },
      "patch_type": {
        "type": "string",
        "enum": ["raw_c_code", "macro_define", "include_header", "ast_re"]
      },
      "required_headers": {
        "type": "array",
        "items": { "type": "string" },
        "description": "C headers required by the snippet."
      },
      "code_content": {
        "type": "string",
        "description": "The clean, valid C99 code snippet to inject. Do

```

```
}
}
}
}
}
}
}
}
```

2. Example OpenAI API Request Payload

When prompting your agent to create a custom shard based on user requests, send your payload structured like this:

JSON



```
{
  "model": "gpt-4o",
  "messages": [
    {
      "role": "system",
      "content": "You are an expert Flipper Zero firmware assistant. Generate cc",
    },
    {
      "role": "user",
      "content": "Create a shard that modifies 'targets/f7/furi_hal/furi_hal_suk",
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "generate_firmware_shard",
        "description": "Generates a valid Flipper Zero firmware shard.",
        "parameters": {
          "type": "object",
          "properties": { ... }
        }
      }
    }
  ]
}
```

```
"tool_choice": {  
  "type": "function",  
  "function": { "name": "generate_firmware_shard" }  
}  
}
```

By forcing `tool_choice` to your shard generator function, OpenAI guarantees that the output will bypass standard conversational text and return a **structured JSON object** matching your schema precisely, which you can then pass straight into your validator.